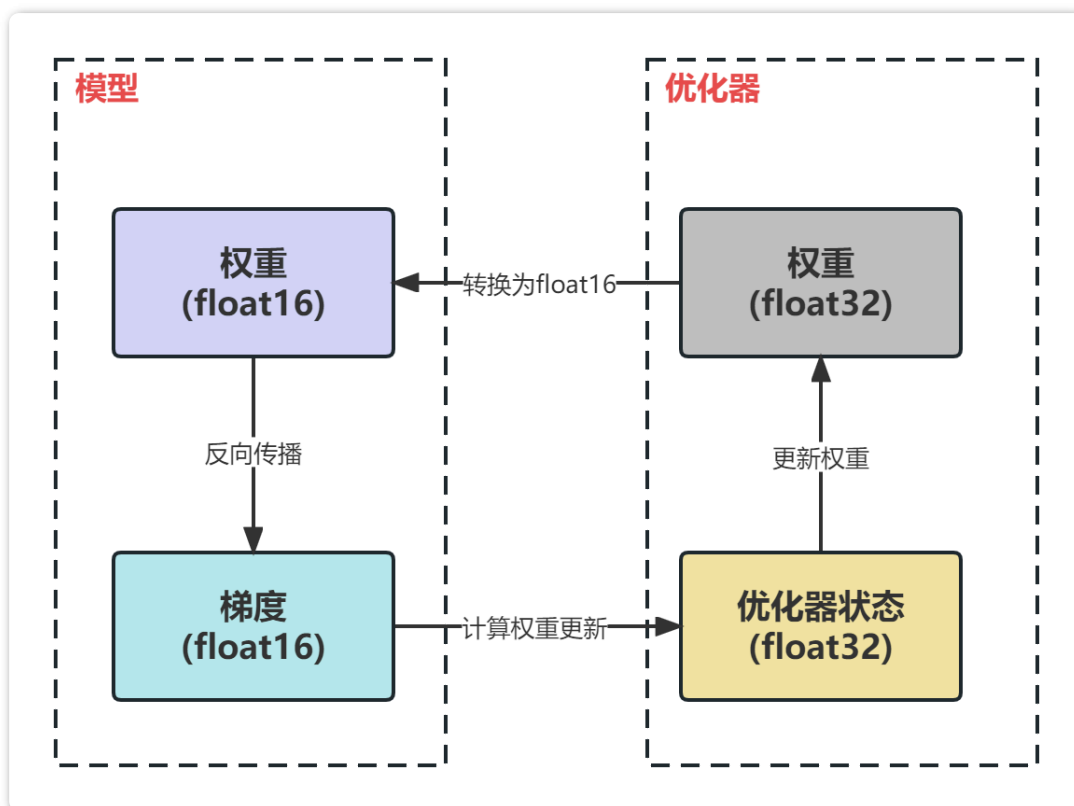


混合精度训练 (Mixed Precision Training)

是一种在深度学习中加速模型训练、减少显存占用并保持模型精度的技术。它通过在训练过程中同时使用**单精度浮点数 (FP32)** 和 **半精度浮点数 (FP16)** 来实现。



一、什么是混合精度？

- **FP32 (单精度)**: 32位浮点数，精度高，但占用内存大、计算慢。
- **FP16 (半精度)**: 16位浮点数，内存占用减半，计算更快（尤其在支持 Tensor Core 的 GPU 上），但数值范围小，容易出现**下溢 (underflow)** * **或*溢出 (overflow)**。

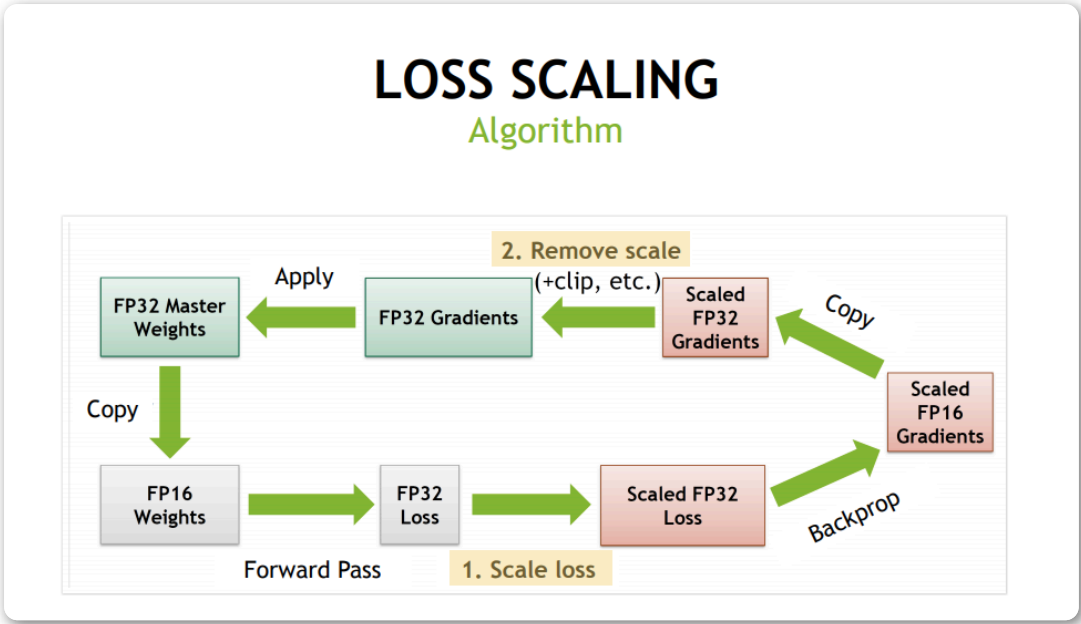
混合精度训练结合两者优势：

- **前向传播和反向传播使用 FP16**：加快计算速度，节省显存。
- **关键部分保留 FP32**：如权重更新、主权重副本 (master weights)，以保证数值稳定性。

二、混合精度训练的优势

优势	说明
加速训练	FP16 计算更快，尤其在 NVIDIA Volta、Ampere 架构的 GPU 上支持 Tensor Cores，可带来 2-3 倍速度提升。
减少显存占用	参数、梯度、激活值使用 FP16 存储，显存占用约减少 50%。
支持更大 batch size 或模型	显存节省允许训练更大的模型或使用更大的 batch size。
保持模型精度	通过保留关键部分的 FP32，避免精度损失。

三、混合精度的关键技术



- 1 主权重（Master Weights）
 - 模型参数在 FP16 中进行前向和反向传播。
 - 但维护一份 FP32 的“主权重”用于参数更新。
 - 每次更新时：FP16 梯度 → 转为 FP32 → 更新 FP32 主权重 → 复制回 FP16 模型。
- 2 损失缩放（Loss Scaling）

- FP16 的数值范围有限 (约 $6e-5 \sim 6.5e4$)，小梯度可能变为 0 (下溢)。
- 解决方案：在反向传播前将损失乘以一个缩放因子 (如 128、512、1024)，使梯度变大；反向传播后，再将梯度除以该因子。
- 可使用**动态损失缩放**：自动调整缩放因子。

四、如何使用混合精度训练？

1. 使用 PyTorch (推荐使用 `torch.cuda.amp`)

```
1 from torch.cuda.amp import autocast, GradScaler
2
3 model = model.cuda()
4 optimizer = torch.optim.Adam(model.parameters())
5 scaler = GradScaler()
6
7 for data, target in dataloader:
8     optimizer.zero_grad()
9
10    # 前向过程使用 autocast
11    with autocast():
12        output = model(data)
13        loss = loss_fn(output, target)
14
15    # 反向传播使用缩放
16    scaler.scale(loss).backward()
17    scaler.step(optimizer)
18    scaler.update() # 更新缩放因子
```

2. 使用 TensorFlow/Keras

```
1 from tensorflow.keras.mixed_precision import
  Policy, set_global_policy
2
3 # 设置全局策略为混合精度
4 set_global_policy(Policy('mixed_float16'))
5
6 model = YourModel()
7 model.compile(optimizer='adam',
  loss='sparse_categorical_crossentropy', metrics=
  ['accuracy'])
8
9 # 注意：输出层通常仍用 float32（避免 softmax 精度损失）
```

五、注意事项

① 并非所有层都适合 FP16

- Softmax、BatchNorm、LayerNorm 等对数值敏感，需注意实现。
- 在 Keras 中，最后一层通常保持 `float32`。

② 硬件支持

- 推荐使用 NVIDIA GPU（如 Tesla V100, A100, RTX 30xx/40xx）。
- 需要 CUDA 和 cuDNN 支持。

③ 数值稳定性

- 损失突然变为 NaN？尝试降低损失缩放因子或启用动态缩放。
- 某些优化器（如 LARS）在 FP16 下需特别处理。

④ 模型收敛性

- 多数情况下精度与 FP32 相当，但某些模型可能需要调参。

六、适用场景

✔ 推荐使用混合精度：

- 大模型训练（如 Transformer、BERT、ResNet）
- 显存受限
- 使用现代 GPU（支持 Tensor Core）

✘ 不推荐或需谨慎：

- 嵌入式设备或不支持 FP16 的硬件
- 对数值精度极度敏感的任务（如科学计算）

七、性能对比示例（大致）

精度	显存占用	训练速度	模型精度
FP32	100%	1x	高
FP16	~50%	1.5~3x	略低（但可通过技巧保持）
混合精度	~60%	2~3x	接近 FP32

总结

混合精度训练是现代深度学习训练的**标配技术**之一，尤其在大模型时代，能显著提升训练效率。通过 `PyTorch AMP` 或 `TensorFlow Mixed Precision` API，可以轻松集成，无需修改模型结构。

✔ **建议：**只要硬件支持，几乎所有的训练任务都可以尝试开启混合精度！